

PATRONES DE CREACIÓN

Guía práctica: Patrón Builder

Explicación, ejemplo resuelto y ejercicio propuesto



Objetivo

Comprender qué problema resuelve Builder, cómo se implementa y cómo se utiliza para construir objetos complejos paso a paso.

Luis Miguel Garay Gallastegui · Universidad San Pablo CEU

1. ¿Qué problema resuelve Builder?

Builder es un patrón de creación pensado para situaciones en las que construir un objeto empieza a ser complicado porque tiene muchas partes, muchas opciones o varios pasos. En lugar de crear todo el objeto de golpe, Builder permite construirlo paso a paso.

Imaginemos una pantalla de una aplicación. Puede tener título, menú, contenido, botón principal, pie de página y otras opciones de configuración.

```
Pantalla pantalla = new Pantalla(
    "Inicio",
    "Menú principal",
    "Bienvenido a la aplicación",
    "Entrar",
    "© 2026"
);
```

Con pocos parámetros puede ser manejable. Pero cuando el objeto crece, el constructor puede convertirse en algo difícil de leer y mantener:

```
new Pantalla(
    "Inicio",
    true,
    false,
    "azul",
    1200,
    800,
    "Menú",
    "Entrar",
    null,
    true,
    false
    // ... muchas opciones más
);
```

Idea clave: Builder evita tener que construir un objeto complejo “de una sola vez”. La construcción se organiza en pasos claros.

Este enfoque coincide con la idea vista en clase: cuando aumenta la complejidad de la creación de un objeto, puede resultar difícil gestionarla desde el constructor y conviene delegar esa construcción en otra clase.

2. Builder

La idea de Builder se puede resumir así:

BUILDER = construir un objeto complejo paso a paso, separando el proceso de construcción del objeto final.

Podemos imaginarlo como la construcción de una casa:

1. Construir cimientos
2. Construir paredes
3. Colocar tejado
4. Colocar ventanas
5. Entregar la casa terminada

Con un objeto software ocurre lo mismo. No necesitamos tener el objeto completo desde el primer momento: lo vamos montando poco a poco hasta obtener el resultado final.

3. ¿Qué significa “separar la construcción de la representación”?

Esta frase puede parecer complicada, pero la idea es sencilla. Una cosa es el proceso que seguimos para construir algo y otra cosa es el resultado concreto que obtenemos.

Por ejemplo, podemos construir distintas pantallas siguiendo siempre los mismos pasos:

```

Título
↓
Cabecera
↓
Contenido
↓
Botón principal
↓
Pantalla terminada
  
```

Ese es el proceso de construcción. Sin embargo, usando esos mismos pasos podemos obtener una pantalla de Login, una pantalla de Perfil o una pantalla de Ajustes. El proceso es parecido, pero la representación final cambia.

Para recordar: el proceso responde a “¿qué pasos sigo?” y la representación responde a “¿qué objeto concreto obtengo al final?”.

4. Las cuatro piezas de Builder

Para resolver los ejercicios, conviene identificar siempre estas cuatro piezas:

Elemento	Pregunta sencilla	Responsabilidad
Product	¿Qué objeto quiero construir?	Es el objeto final.
Builder	¿Qué pasos hacen falta?	Define las operaciones de construcción.
ConcreteBuilder	¿Cómo hago cada paso en este caso concreto?	Implementa los pasos y va montando el producto.
Director	¿En qué orden ejecuto los pasos?	Coordina el proceso de construcción.

5. Ejemplo resuelto: construir pantallas de una aplicación

Vamos a construir una pantalla de Login utilizando Builder. El objetivo no es crear una interfaz gráfica real, sino representar de forma sencilla sus componentes para entender el patrón.

5.1. Product: la clase Pantalla

El Product es el objeto que queremos obtener al final. La clase Pantalla guarda los elementos de la interfaz, pero no decide cómo se construye.

```
class Pantalla {  
  
    private String titulo;  
    private String menu;  
    private String contenido;  
    private String boton;  
  
    public void setTitulo(String titulo) {  
        this.titulo = titulo;  
    }  
  
    public void setMenu(String menu) {  
        this.menu = menu;  
    }  
  
    public void setContenido(String contenido) {  
        this.contenido = contenido;  
    }  
  
    public void setBoton(String boton) {  
        this.boton = boton;  
    }  
  
    public void mostrar() {  
        System.out.println("Título: " + titulo);  
        System.out.println("Menú: " + menu);  
        System.out.println("Contenido: " + contenido);  
        System.out.println("Botón: " + boton);  
    }  
}
```

Importante: Pantalla representa el resultado final. No contiene la lógica que decide qué título, menú, contenido o botón debe tener cada tipo de pantalla.

5.2. Builder: definir los pasos

Creamos una interfaz que indica qué pasos son necesarios para construir una pantalla. Todavía no decidimos qué valores tendrá cada elemento.

```
interface PantallaBuilder {  
  
    void construirTitulo();  
  
    void construirMenu();  
  
    void construirContenido();  
  
    void construirBoton();  
  
    Pantalla getPantalla();  
}
```

Traducido: cualquier clase que quiera actuar como constructor de pantallas deberá saber construir el título, el menú, el contenido y el botón, y deberá poder devolver la pantalla terminada.

5.2.1. ¿Por qué usamos una interfaz Builder?

Porque queremos que todos los constructores concretos compartan el mismo contrato. Si LoginBuilder, PerfilBuilder o AjustesBuilder implementan PantallaBuilder, sabemos que todos ofrecen los mismos pasos de construcción.

- El Director no necesita conocer qué Builder concreto está utilizando.
- El Director puede trabajar siempre con el tipo general PantallaBuilder.
- Podemos cambiar un Builder por otro sin modificar el proceso general de construcción.

```
PantallaBuilder builder = new LoginBuilder();  
DirectorPantalla director = new DirectorPantalla(builder);
```

Atención: la interfaz Builder dice QUÉ PASOS existen; cada ConcreteBuilder decide CÓMO realiza esos pasos. Así, el Director puede ordenar los pasos sin conocer los detalles.

5.3. ConcreteBuilder: LoginBuilder

Ahora sí decidimos cómo será cada parte para un caso concreto: una pantalla de Login.

```
class LoginBuilder implements PantallaBuilder {

    private Pantalla pantalla;

    public LoginBuilder() {
        pantalla = new Pantalla();
    }

    @Override
    public void construirTitulo() {
        pantalla.setTitulo("Iniciar sesión");
    }

    @Override
    public void construirMenu() {
        pantalla.setMenu("Sin menú");
    }

    @Override
    public void construirContenido() {
        pantalla.setContenido(
            "Usuario: _____ Contraseña: _____"
        );
    }

    @Override
    public void construirBoton() {
        pantalla.setBoton("ENTRAR");
    }

    @Override
    public Pantalla getPantalla() {
        return pantalla;
    }
}
```

LoginBuilder está diciendo cómo se construye cada parte concreta:

- Título → “Iniciar sesión”.
- Menú → “Sin menú”.
- Contenido → usuario y contraseña.
- Botón → “ENTRAR”.

5.4. Director: controlar el proceso

El Director no decide qué texto tendrá la pantalla. Su responsabilidad es controlar el proceso: qué pasos se ejecutan y en qué orden.

```
class DirectorPantalla {

    private PantallaBuilder builder;

    public DirectorPantalla(PantallaBuilder builder) {
        this.builder = builder;
    }

    public void construirPantalla() {
        builder.construirTitulo();
        builder.construirMenu();
        builder.construirContenido();
        builder.construirBoton();
    }
}
```

Director = sabe QUÉ PASOS ejecutar y en qué orden. ConcreteBuilder = sabe CÓMO realizar cada paso.

5.5. Main: utilizar el patrón

```
public class Main {

    public static void main(String[] args) {

        PantallaBuilder builder = new LoginBuilder();

        DirectorPantalla director =
            new DirectorPantalla(builder);

        director.construirPantalla();

        Pantalla pantalla = builder.getPantalla();

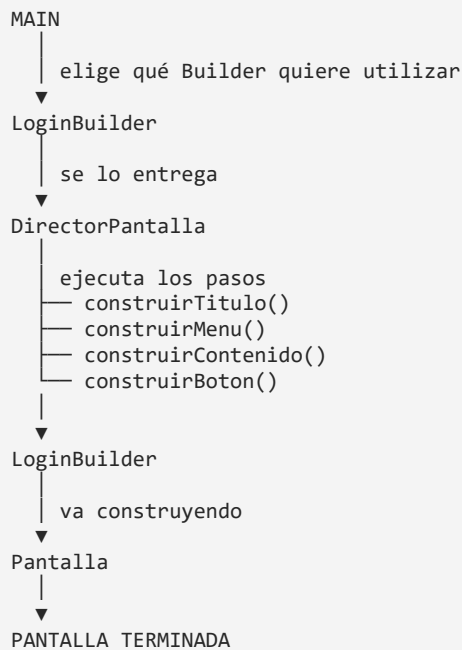
        pantalla.mostrar();
    }
}
```

La salida sería aproximadamente:

```
Título: Iniciar sesión
Menú: Sin menú
Contenido: Usuario: _____ Contraseña: _____
Botón: ENTRAR
```

6. ¿Qué está ocurriendo realmente?

El recorrido completo puede entenderse así:



Qué debe entender el alumno: el Director organiza el proceso, el Builder concreto construye cada parte y el Product es el objeto que recibimos al final.

7. ¿Qué beneficios aporta Builder?

- Evita constructores enormes y difíciles de entender cuando un objeto tiene muchos datos u opciones.
- Divide la creación del objeto en pasos claros y fáciles de seguir.
- Separa la clase del producto de la lógica utilizada para construirlo.
- Permite reutilizar un mismo proceso de construcción para obtener resultados diferentes.
- Cada ConcreteBuilder puede construir una representación distinta del mismo tipo de producto.
- El Director puede ocultar al cliente los detalles de cómo se ejecuta la construcción.

8. Ejercicio propuesto: Constructor de pantallas de usuario

Una aplicación necesita generar distintos tipos de pantallas. Todas las pantallas tendrán cuatro elementos: título, cabecera, contenido y botón principal.

Se desea construir al menos dos tipos de pantalla:

- Pantalla de PERFIL.
- Pantalla de AJUSTES.

Objetivo del ejercicio: implementar la solución utilizando correctamente el patrón Builder. No basta con conseguir la salida: la estructura del patrón debe respetarse.

8.1. Requisito 1 — Crear el Product

Crea una clase Pantalla con los siguientes atributos:

```
String titulo;  
String cabecera;  
String contenido;  
String botonPrincipal;
```

Incluye los métodos set necesarios y un método:

```
public void mostrar()
```

que muestre en consola todos los elementos de la pantalla.

8.2. Requisito 2 — Crear el Builder

Crea la interfaz PantallaBuilder con los siguientes métodos:

```
void construirTitulo();  
void construirCabecera();  
void construirContenido();  
void construirBoton();  
Pantalla getPantalla();
```

8.3. Requisito 3 — Crear los ConcreteBuilder

Debes crear dos clases que implementen PantallaBuilder:

- PerfilBuilder.
- AjustesBuilder.

PerfilBuilder

Debe construir una pantalla con estos datos aproximados:

```
Título:  
Mi perfil  
  
Cabecera:  
Datos del usuario  
  
Contenido:  
Nombre - Email - Fotografía  
  
Botón:  
EDITAR PERFIL
```

AjustesBuilder

Debe construir una pantalla con estos datos aproximados:

```

Título:
Ajustes

Cabecera:
Configuración de la aplicación

Contenido:
Idioma - Tema - Notificaciones

Botón:
GUARDAR CAMBIOS

```

8.4. Requisito 4 — Crear el Director

Crea una clase `DirectorPantalla`. El Director debe recibir un objeto de tipo `PantallaBuilder`, no un builder concreto.

```
private PantallaBuilder builder;
```

Debe incluir un método `construirPantalla()` que ejecute los pasos en este orden:

```

builder.construirTitulo();
builder.construirCabecera();
builder.construirContenido();
builder.construirBoton();

```

8.5. Requisito 5 — Probarlo desde Main

En Main construye primero una pantalla de Perfil:

```

PantallaBuilder perfilBuilder =
    new PerfilBuilder();

DirectorPantalla director =
    new DirectorPantalla(perfilBuilder);

```

Haz que el Director construya la pantalla y, una vez terminada, recupérela mediante `getPantalla()` y muéstrala. A continuación, repite el proceso utilizando `AjustesBuilder`.

8.6. Resultado aproximado

```

-----
MI PERFIL
-----
Datos del usuario

Nombre - Email - Fotografía

[ EDITAR PERFIL ]

```

```

-----
AJUSTES
-----
Configuración de la aplicación

Idioma - Tema - Notificaciones

[ GUARDAR CAMBIOS ]

```

9. Restricciones del ejercicio

1. Main no debe asignar directamente los atributos de Pantalla.
2. PerfilBuilder y AjustesBuilder deben implementar PantallaBuilder.
3. DirectorPantalla debe trabajar con PantallaBuilder, no directamente con PerfilBuilder o AjustesBuilder.
4. La pantalla debe construirse paso a paso mediante los métodos definidos en el Builder.
5. Al finalizar la construcción, el objeto Pantalla debe recuperarse mediante getPantalla().
6. La lógica que decide el contenido concreto de cada pantalla debe estar en su ConcreteBuilder correspondiente.

10. Pregunta de comprensión

Además del código, responde brevemente a la siguiente pregunta:

Pregunta: ¿Por qué utilizamos Builder en lugar de crear directamente un objeto Pantalla con un constructor que reciba todos sus atributos?

Una respuesta correcta debería mencionar que Builder resulta útil cuando la construcción tiene varios pasos o configuraciones y que permite separar esa lógica de construcción de la clase que representa el producto final.

11. Checklist antes de entregar

- ☐ He creado la clase Product (Pantalla).
- ☐ He creado la interfaz Builder (PantallaBuilder).
- ☐ He creado al menos dos ConcreteBuilder.
- ☐ Cada ConcreteBuilder crea y configura su propia Pantalla.
- ☐ He creado DirectorPantalla.
- ☐ El Director trabaja con PantallaBuilder.
- ☐ El objeto se construye mediante varios pasos.
- ☐ Obtengo el objeto final mediante getPantalla().
- ☐ Main no construye directamente los elementos internos de Pantalla.
- ☐ Puedo explicar qué responsabilidad tiene Product, Builder, ConcreteBuilder y Director.

12. Chuleta final: Singleton vs Factory Method vs Builder

Patrón	Pregunta principal	Idea para recordar
Singleton	¿Cuántas instancias pueden existir?	Un único objeto.
Factory Method	¿Quién decide qué objeto concreto se crea?	Delegar la elección del tipo de objeto.
Builder	¿Cómo construyo paso a paso un objeto complejo?	Separar y organizar el proceso de construcción.

En una frase: Singleton controla CUÁNTOS objetos existen; Factory Method controla QUIÉN decide QUÉ objeto crear; Builder organiza CÓMO construir un objeto complejo paso a paso.

Material elaborado a partir de la teoría de patrones de creación impartida en la asignatura.